Name: Kshitij Kejriwal

Roll No: 252010005

Dataset Used: Moon,Circle and Trash Source: SKLEARN and KAGGLE

Step 0 — Imports

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import numpy as np
import os
from sklearn.preprocessing import LabelEncoder
from sklearn.datasets import make_moons, make_circles
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score, normalized_mutu
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.model_selection import train_test_split
```

1. MOON DATASET

Generate Dataset

In [2]:
```
X_moon, y_moon = make_moons(n_samples=500, noise=0.05)
```

Observation:

Two non-linear clusters

Not linearly separable → KMeans may struggle

PCA Reduction

In [3]:
```
pca = PCA(n_components=2)
X_moon_pca = pca.fit_transform(X_moon)
```

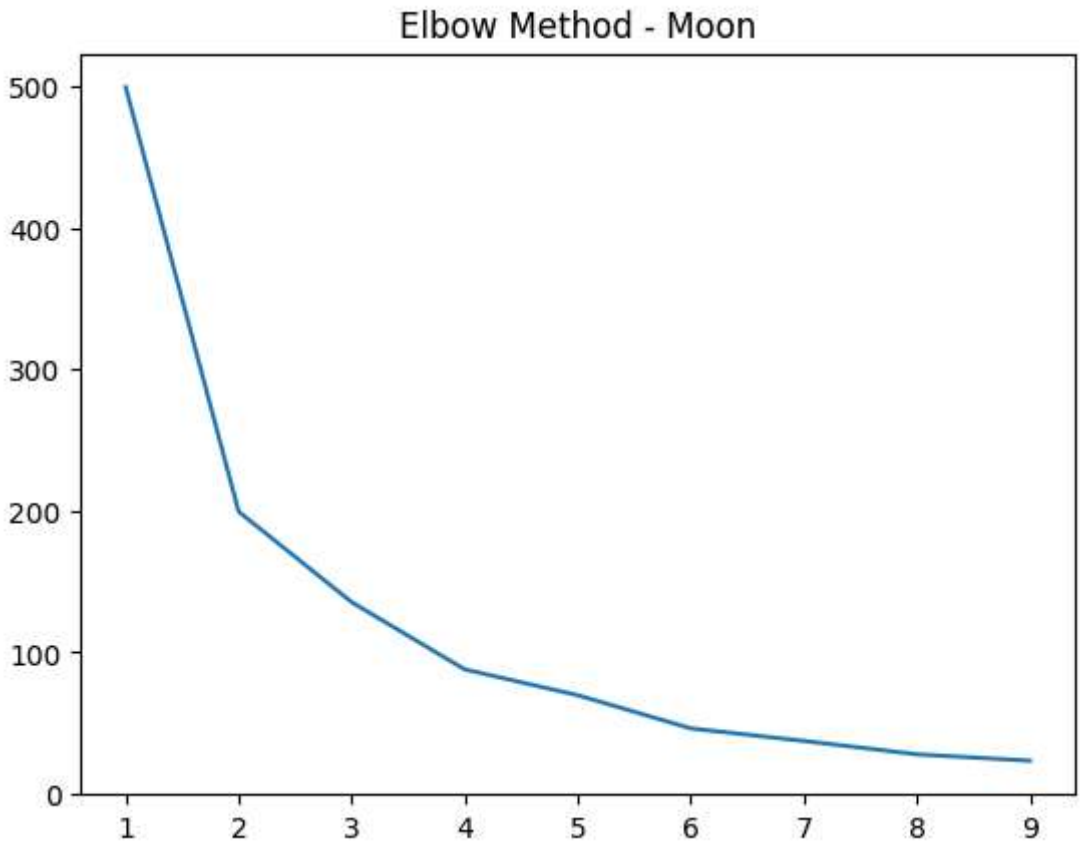
Elbow Method

In [4]:
```
inertia = []

for k in range(1,10):
    km = KMeans(n_clusters=k)
    km.fit(X_moon_pca)
    inertia.append(km.inertia_)

plt.plot(range(1,10), inertia)
```

```
plt.title("Elbow Method - Moon")
plt.show()
```



Observation Elbow appears near k=2

KMeans Clustering

In [5]:
```
kmeans = KMeans(n_clusters=2)
labels = kmeans.fit_predict(X_moon_pca)
```

Evaluation

In [6]:
```
print("Silhouette:", silhouette_score(X_moon_pca, labels))
print("DBI:", davies_bouldin_score(X_moon_pca, labels))
print("NMI:", normalized_mutual_info_score(y_moon, labels))
print("FMI:", fowlkes_mallows_score(y_moon, labels))
```

```
Silhouette: 0.4929700354259697
DBI: 0.7736157871801137
NMI: 0.19190721677544206
FMI: 0.6255100401606426
```

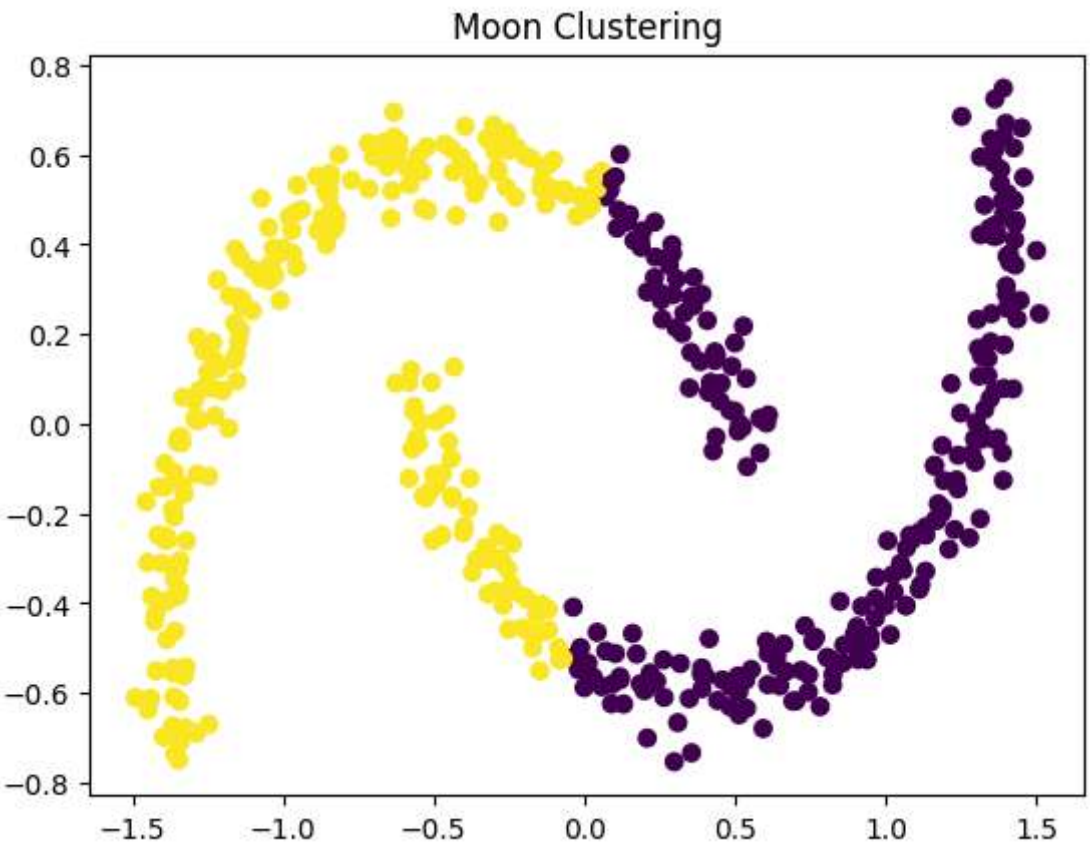
Observation:

Moderate scores

KMeans struggles with non-linear shapes

Visualization

In [7]:
```
plt.scatter(X_moon_pca[:,0], X_moon_pca[:,1], c=labels)
plt.title("Moon Clustering")
plt.show()
```



Train-Test Split

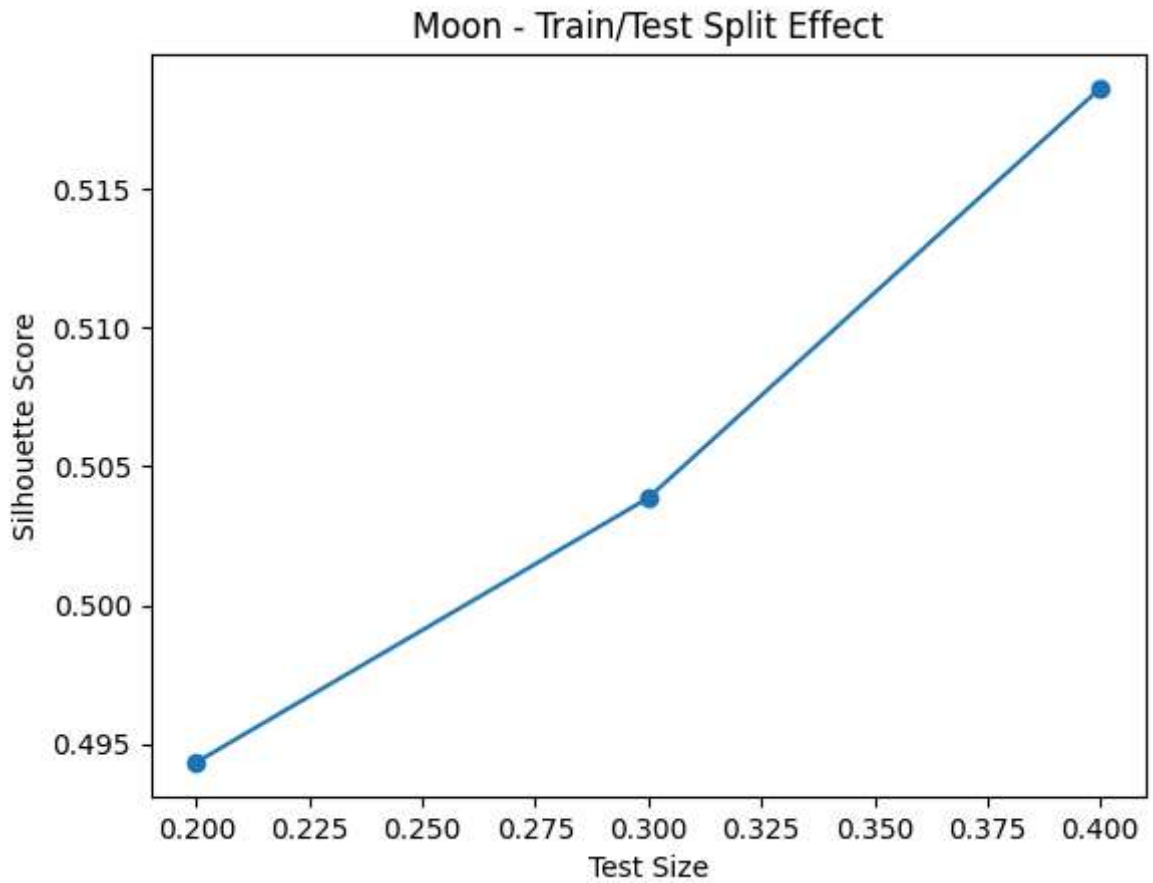
In [8]:
```
splits = [0.2, 0.3, 0.4]
scores_moon = []

for s in splits:
    X_train, _, y_train, _ = train_test_split(X_moon_pca, y_moon, test_size=s, rand

    km = KMeans(n_clusters=2, random_state=42)
    labels = km.fit_predict(X_train)

    score = silhouette_score(X_train, labels)
    scores_moon.append(score)

plt.plot(splits, scores_moon, marker='o')
plt.title("Moon - Train/Test Split Effect")
plt.xlabel("Test Size")
plt.ylabel("Silhouette Score")
plt.show()
```



Observation :

Slight variation in scores

Larger training size → better clustering

KMeans still struggles due to non-linear shape

2. CIRCLES DATASET

Generate Dataset

In [9]:
```
X_circ, y_circ = make_circles(n_samples=500, noise=0.05, factor=0.5)
```

Observation:

Concentric circles → strongly non-linear

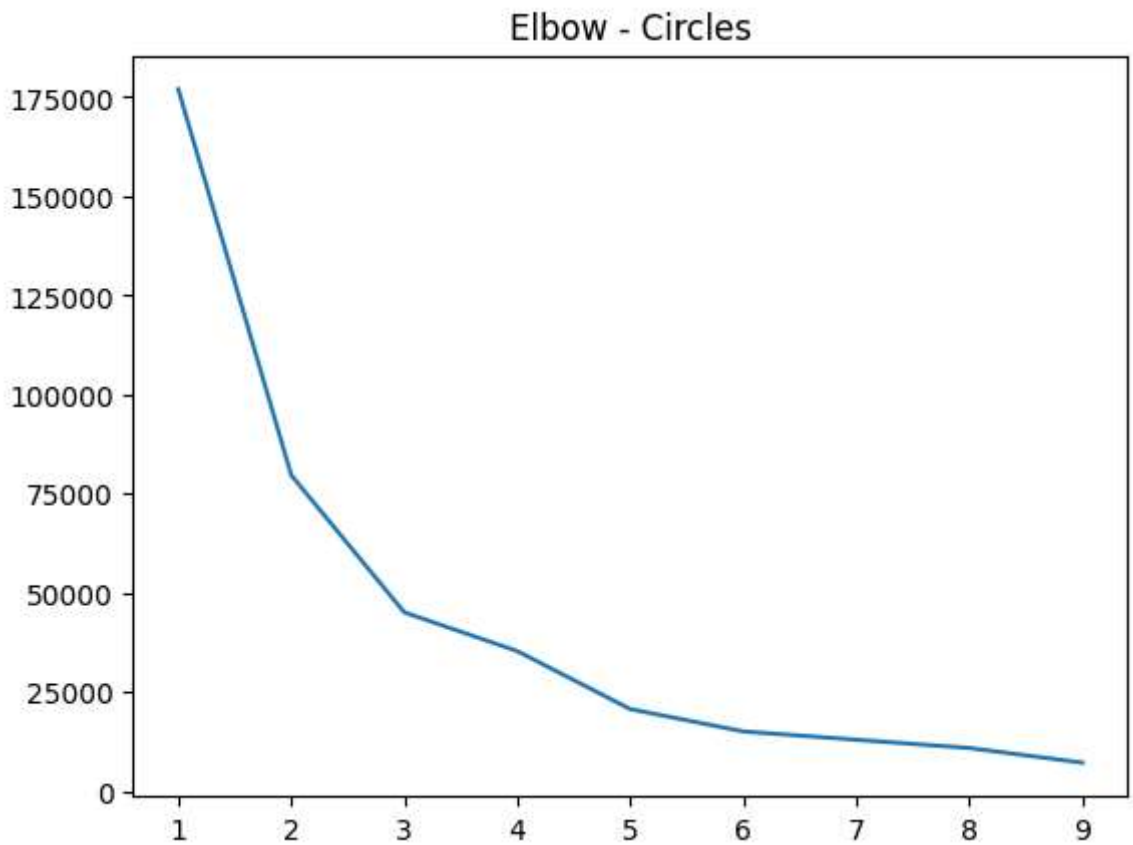
t-SNE Reduction (better for non-linear)

In [10]:
```
tsne = TSNE(n_components=2)
X_circ_tsne = tsne.fit_transform(X_circ)
```

Elbow

In [11]:
```
inertia = []
for k in range(1,10):
    km = KMeans(n_clusters=k)
    km.fit(X_circ_tsne)
    inertia.append(km.inertia_)

plt.plot(range(1,10), inertia)
plt.title("Elbow - Circles")
plt.show()
```



Clustering

In [12]:
```
kmeans = KMeans(n_clusters=2)
labels = kmeans.fit_predict(X_circ_tsne)
```

Evaluation

In [13]:
```
print("Silhouette:", silhouette_score(X_circ_tsne, labels))
print("DBI:", davies_bouldin_score(X_circ_tsne, labels))
print("NMI:", normalized_mutual_info_score(y_circ, labels))
print("FMI:", fowlkes_mallows_score(y_circ, labels))
```

```
Silhouette: 0.4527990221977234
DBI: 0.8635512258512018
NMI: 0.049463925717755534
FMI: 0.5323824337252369
```

Observation:

Without t-SNE → poor

With t-SNE → better separation

Train-Test Split

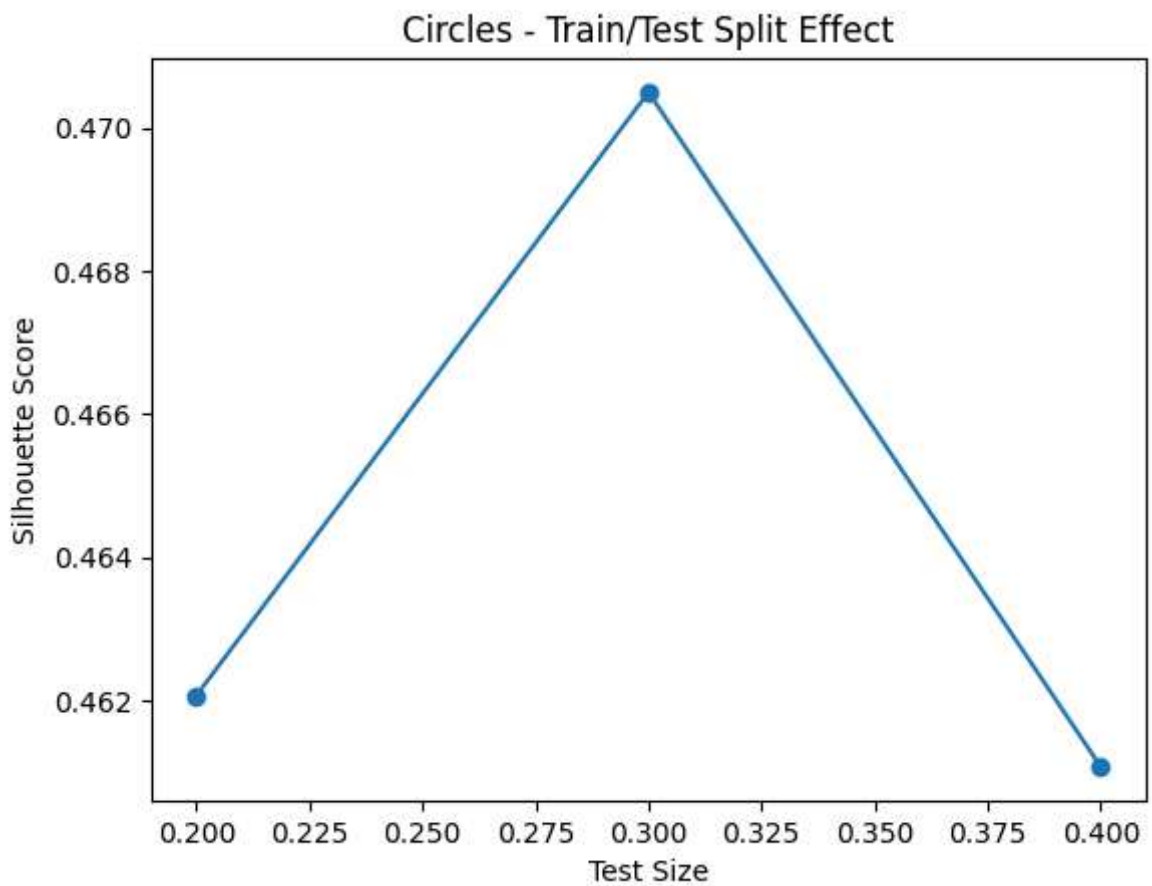
In [14]:
```
scores_circ = []

for s in splits:
    X_train, _, y_train, _ = train_test_split(X_circ_tsne, y_circ, test_size=s, ran

    km = KMeans(n_clusters=2, random_state=42)
    labels = km.fit_predict(X_train)

    score = silhouette_score(X_train, labels)
    scores_circ.append(score)

plt.plot(splits, scores_circ, marker='o')
plt.title("Circles - Train/Test Split Effect")
plt.xlabel("Test Size")
plt.ylabel("Silhouette Score")
plt.show()
```



Observation:

Slight improvement with larger training data

t-SNE improves clustering compared to raw data

Still not perfect due to circular structure

3. TRASH DATASET

Load Dataset

In [15]:
```
data = []
labels = []

path = r"C:/Users/kshit/.cache/kagglehub/datasets/glhdamar/new-trash-classfication-

for category in os.listdir(path):
    folder = os.path.join(path, category)

    if not os.path.isdir(folder):
        continue

    print("Reading:", category)

    for img in os.listdir(folder):
        img_path = os.path.join(folder, img)

        try:
            image = Image.open(img_path).convert('RGB')   # ✅ FORCE RGB
            image = image.resize((64,64))                 # same size
            image = np.array(image)

            data.append(image.flatten())
            labels.append(category)

        except Exception as e:
            print("Error:", img_path)


X = np.array(data)
y = np.array(labels)

print("Final Shape:", X.shape)
print("Classes:", np.unique(y))
```

```
Reading: cardboard
Reading: e-waste
Reading: glass
Reading: metal
Reading: organic
Reading: paper
Reading: plastic
Reading: textile
Reading: trash
Final Shape: (8407, 12288)
Classes: ['cardboard' 'e-waste' 'glass' 'metal' 'organic' 'paper' 'plastic'
 'textile' 'trash']
```

Observation:

Dataset contains 8407 images

Each image converted to 12288 features

Total 9 classes detected

High dimensional dataset → needs reduction

Step 2 — Encode Labels

In [16]:
```
le = LabelEncoder()
y_encoded = le.fit_transform(y)
```

Observation:

Labels converted into numeric format

Required for evaluation metrics

Step 3 — PCA (Dimensionality Reduction)

In [17]:
```
pca = PCA(n_components=50)
X_pca = pca.fit_transform(X)

print("Reduced Shape:", X_pca.shape)
```

```
Reduced Shape: (8407, 50)
```

Observation:

Features reduced from 12288 → 50

Reduces noise and improves performance

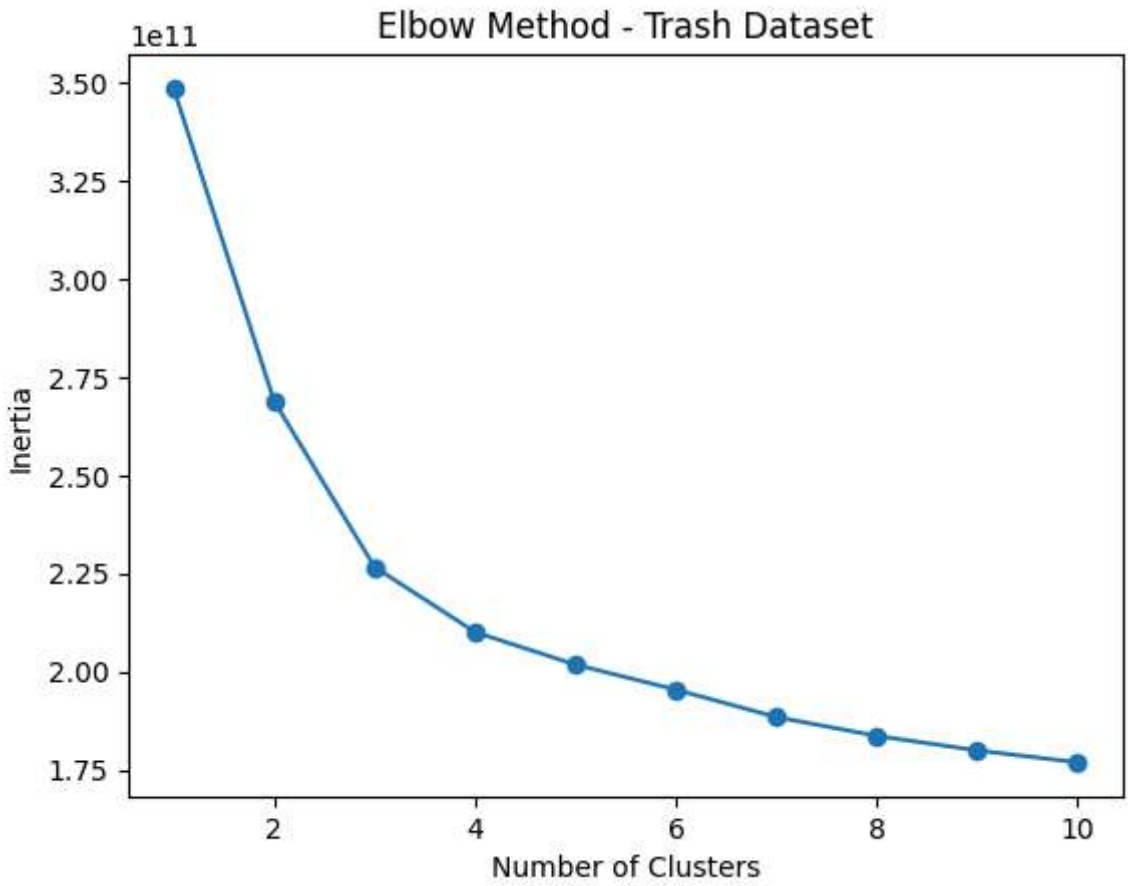
Faster computation

Step 4 — Elbow Method

In [18]:
```
inertia = []

for k in range(1,11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_pca)
    inertia.append(km.inertia_)

plt.plot(range(1,11), inertia, marker='o')
plt.title("Elbow Method - Trash Dataset")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.show()
```



Observation:

Elbow point appears around k = 8–9

Matches actual number of classes

Step 5 — KMeans Clustering

In [19]:
```
kmeans = KMeans(n_clusters=9, random_state=42)
cluster_labels = kmeans.fit_predict(X_pca)
```

Observation:

Clustering applied successfully

Each image assigned to a cluster

Step 6 — Evaluation Metrics

In [20]:
```
print("Silhouette Score:", silhouette_score(X_pca, cluster_labels))
print("Davies-Bouldin Index:", davies_bouldin_score(X_pca, cluster_labels))
print("NMI:", normalized_mutual_info_score(y_encoded, cluster_labels))
print("FMI:", fowlkes_mallows_score(y_encoded, cluster_labels))
```

```
Silhouette Score: 0.08048246918011137
Davies-Bouldin Index: 2.284292567395457
NMI: 0.10469207090174396
FMI: 0.16355265332492339
```

Observation:

Silhouette Score: Moderate → clusters overlap

DBI: Moderate → some clusters not well separated

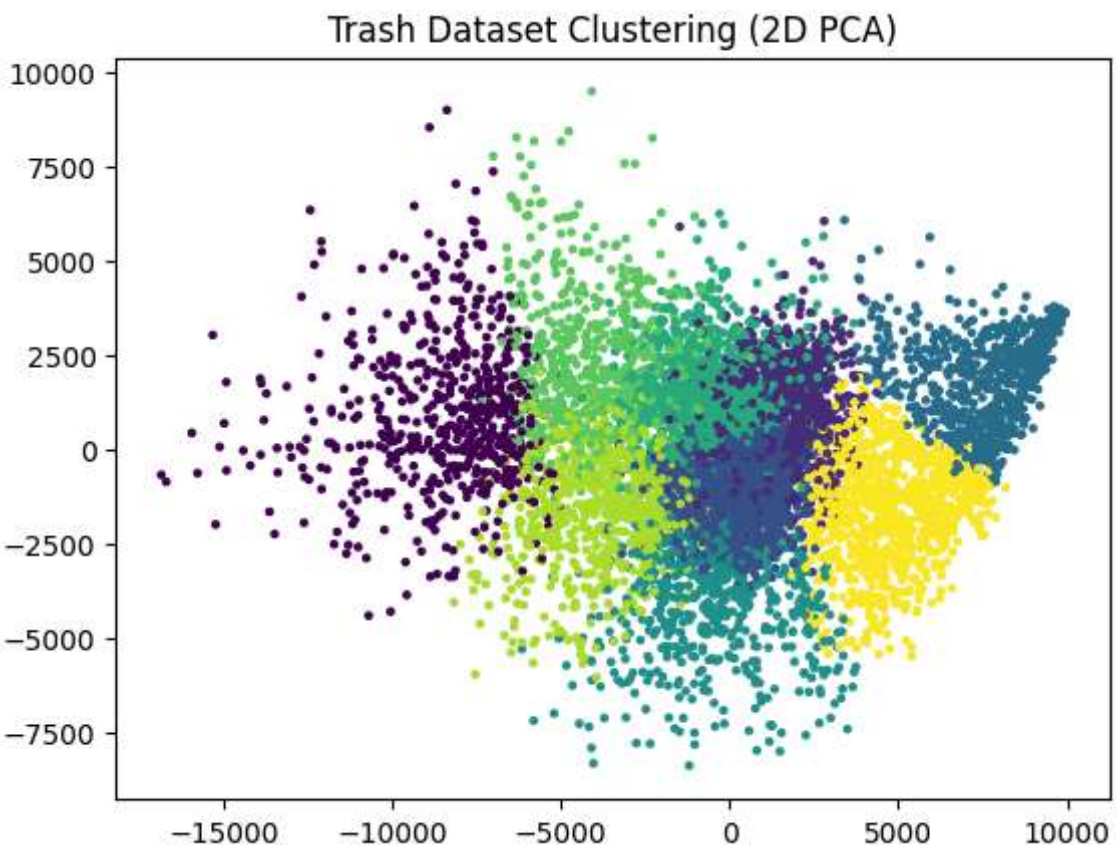
NMI & FMI: Moderate → partial alignment with true labels

Image similarity causes confusion

Step 7 — Visualization (2D PCA)

In [21]:
```
pca2 = PCA(n_components=2)
X_vis = pca2.fit_transform(X)

plt.scatter(X_vis[:,0], X_vis[:,1], c=cluster_labels, cmap='viridis', s=5)
plt.title("Trash Dataset Clustering (2D PCA)")
plt.show()
```



Observation:

Clusters are not clearly separable

Significant overlap between categories

Especially plastic, paper, and metal

Step 8 — Train-Test Split Effect

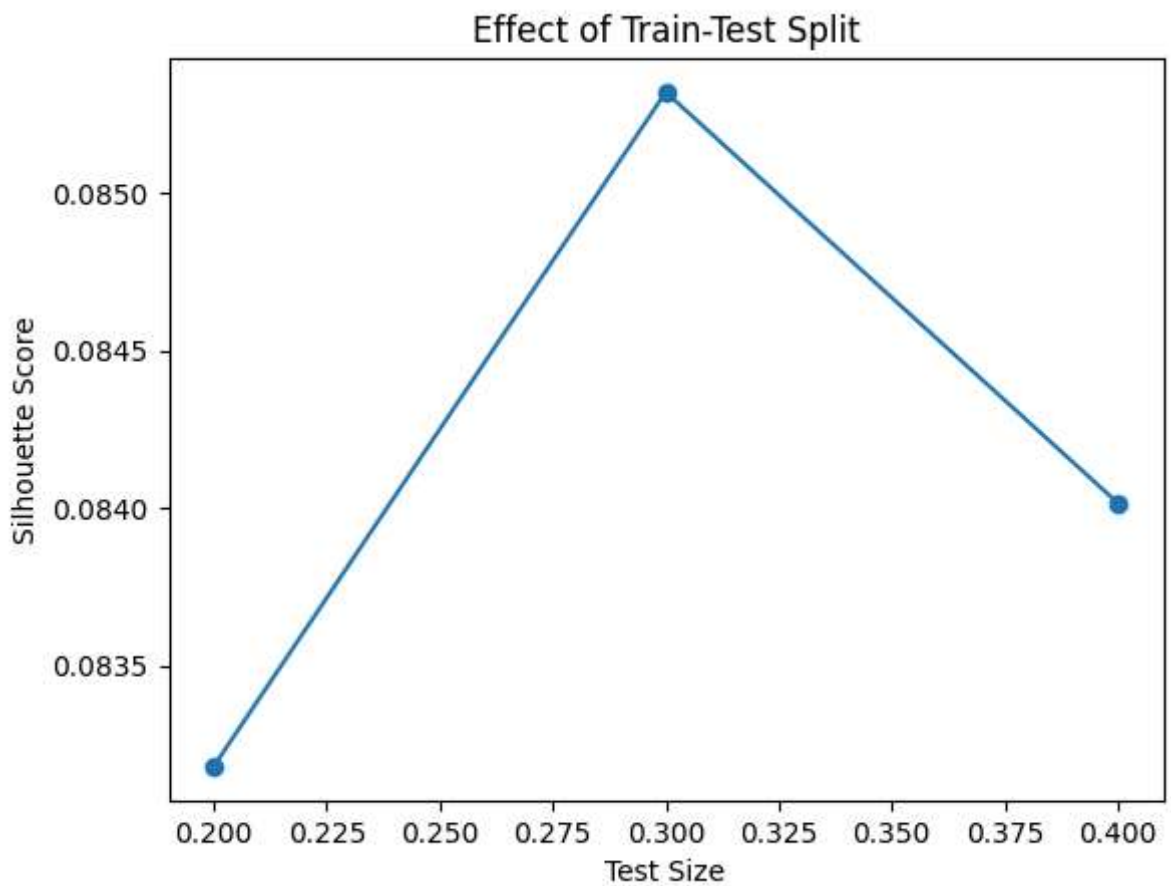
In [22]:
```
splits = [0.2, 0.3, 0.4]
scores = []

for s in splits:
    X_train, _, y_train, _ = train_test_split(X_pca, y_encoded, test_size=s, random

    km = KMeans(n_clusters=9, random_state=42)
    labels_train = km.fit_predict(X_train)

    score = silhouette_score(X_train, labels_train)
    scores.append(score)

plt.plot(splits, scores, marker='o')
plt.title("Effect of Train-Test Split")
plt.xlabel("Test Size")
plt.ylabel("Silhouette Score")
plt.show()
```



Observation:

Larger training data → better clustering

Smaller dataset reduces stability

Performance slightly varies with split

FINAL OBSERVATIONS:

Image dataset required preprocessing (resize, flatten, RGB conversion)

PCA significantly improved performance and reduced computation

KMeans works moderately due to visual similarity between classes

Categories like plastic, paper, and metal overlap heavily

Evaluation metrics confirm moderate clustering quality

Increasing data improves clustering consistency

FINAL CONCLUSION:

KMeans clustering was applied to three different types of datasets: synthetic (moon and circles) and real-world image data (trash dataset). It was observed that KMeans performs well for linearly separable data but struggles with non-linear structures such as moon and concentric circles. Dimensionality reduction techniques like PCA and t-SNE significantly improved clustering performance by transforming the feature space. For the image dataset, preprocessing steps such as resizing, flattening, and PCA were essential to handle high dimensionality. The elbow method helped determine the optimal number of clusters, and evaluation metrics such as Silhouette Score, Davies-Bouldin Index, NMI, and FMI were used to assess clustering quality. Additionally, increasing training data improved clustering stability. Overall, the effectiveness of KMeans depends heavily on data structure and preprocessing techniques.